

Assignment-5
Parallel and Distributed Computing
Submitted by :
Shreyash Tripathi (102303328)

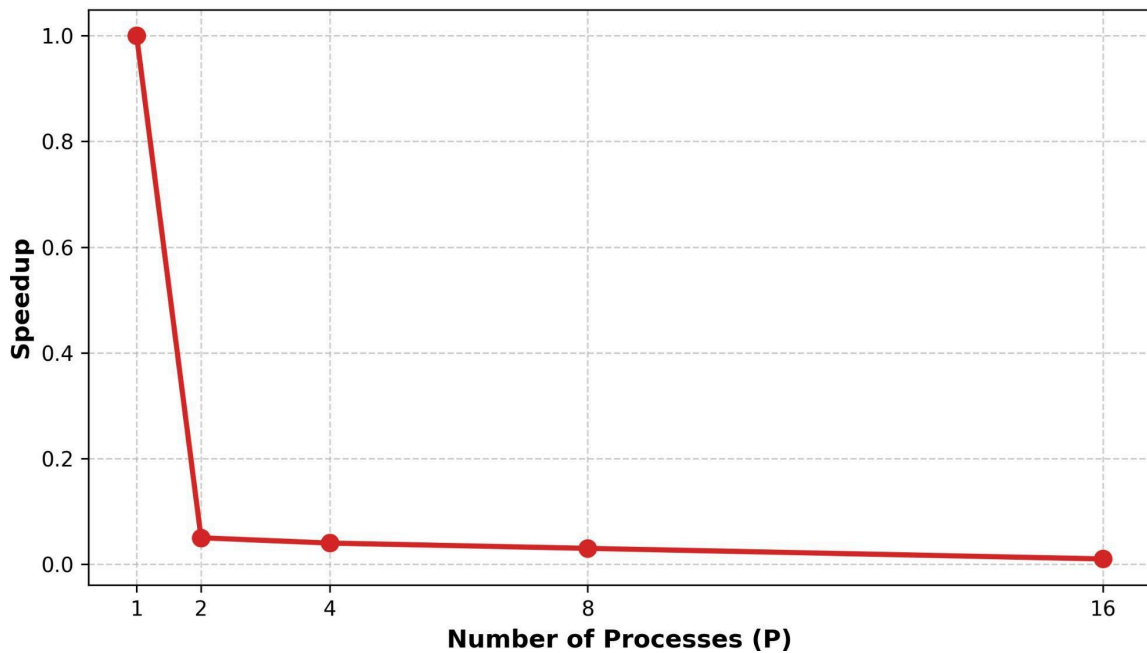
Q1: DAXPY Loop (Double Precision)

Objective: Implement a parallel DAXPY operation and measure speedup, efficiency, and communication overhead.

Performance Data :

Processes	Time (s)	Speedup	Efficiency	Communication Overhead Percentage
1	0.000044	1.00	100.00%	0.00%
2	0.002089	0.05	2.68%	97.50%
4	0.002171	0.04	0.91%	99.29%
8	0.002711	0.03	0.38%	99.70%
16	0.007558	0.01	0.05%	99.96%

Q1: DAXPY Speedup (Memory-Bound)



Analysis & Key Inferences :

The DAXPY experiment serves as a classic demonstration of a memory bound parallel workload where communication dominates.

- **Inference 1: The Parallelization Threshold is Unmet.** The data starkly illustrates that simply dividing a task among multiple processors actively harms performance if the dataset is too small. For a vector size of 65,536 elements, the mathematical operation takes fractions of a millisecond (0.000044s). The overhead of packaging and routing this data via MPI_Scatter and MPI_Gather entirely eclipses the computation time.
- **Inference 2: Negative Scaling via Overhead Dominance.** Communication overhead consistently exceeds 97%, peaking at 99.96% at 16 processes. The system spends its time moving data through the OS stack rather than computing. The negative scaling (speedup dropping to 0.01) proves that adding synchronization barriers to a lightweight task acts as a strict bottleneck.
- **System Design Takeaway:** To achieve positive speedup in distributed vector arithmetic, the computation to communication ratio must be massive. The dataset size must be scaled up drastically to hide network latency.

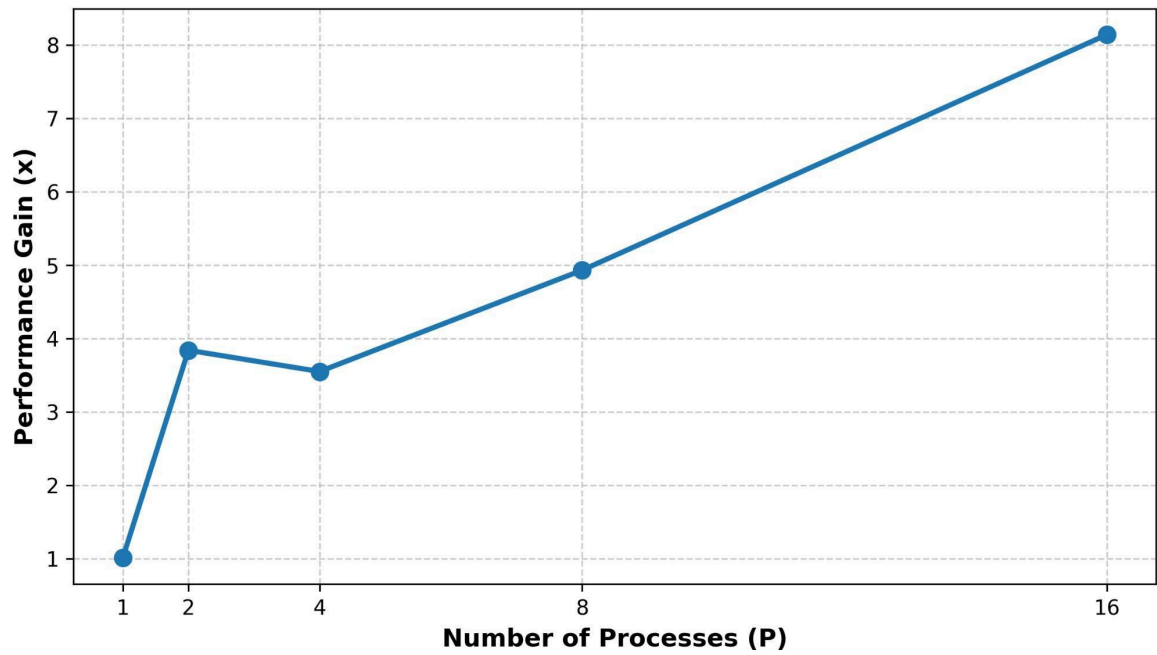
Q2: The Broadcast Race (Linear vs. Tree)

Objective: Compare manual point to point loops against optimized collective MPI_Bcast for a large array.

Performance Data:

Processes	MyBcast (Linear)	MPI_Bcast (Tree)	Performance Gain
2	0.054806 s	0.014264 s	3.84x
4	0.109629 s	0.030867 s	3.55x
8	0.287351 s	0.058255 s	4.93x
16	1.070578 s	0.131504 s	8.14x

Q2: MPI_Bcast Performance Gain Multiplier



Analysis & Key Inferences:

This experiment quantifies the critical importance of algorithmic complexity in network topologies.

- **Inference 1: Linear Topologies Induce Root Bottlenecks.** The manual MyBcast implementation relies on a linear $O(P)$ distribution model. As process counts scale to 16, the execution time balloons to 1.07 seconds. This allows us to infer that Rank 0 suffers from severe message serialization—the hardware can only push one `MPI_Send` onto the network at a time.
- **Inference 2: Tree Topologies Exploit Distributed Bandwidth.** `MPI_Bcast` leverages an $O(\log(P))$ tree topology. By turning receiving nodes into secondary senders, it eliminates the root bottleneck. The performance gap widens as processes increase, culminating in an 8.14x advantage at 16 processes.
- **System Design Takeaway:** Programmers should never attempt to manually reconstruct collective operations for large datasets. Built in MPI primitives are deeply optimized at the middleware layer to maintain scalability.

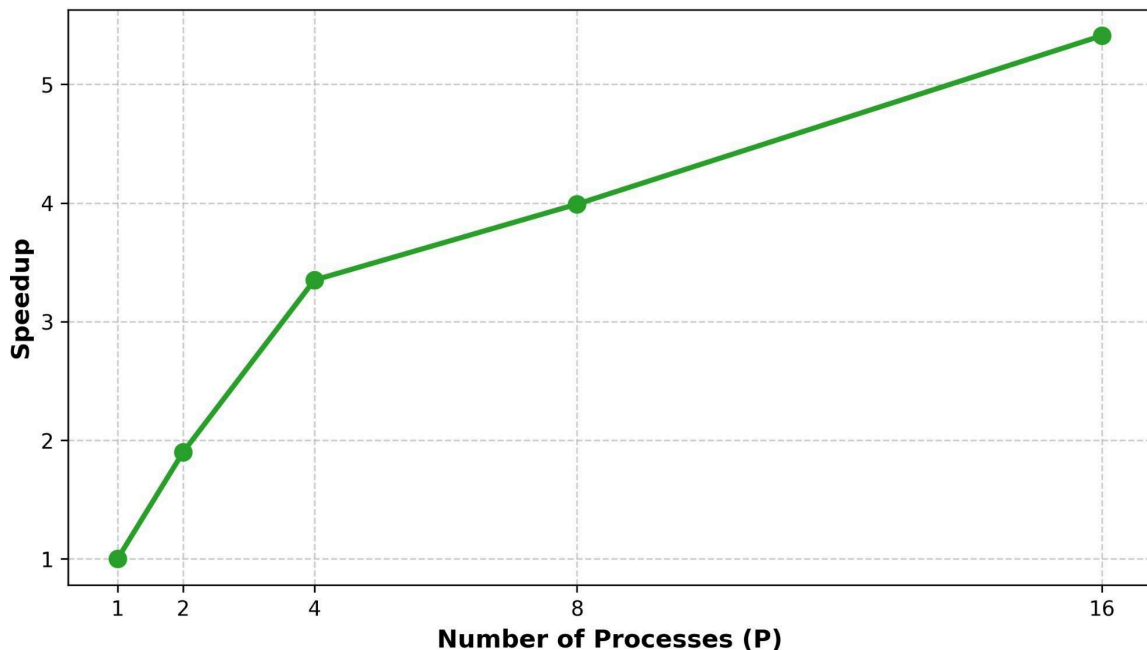
Q3: Distributed Dot Product & Amdahl's Law

Objective: Calculate the dot product of a large vector using local generation, `MPI_Bcast`, and `MPI_Reduce`.

Performance Data:

Processes	Time (s)	Speedup	Efficiency	Comm %
1	3.852155	1.00	100.00%	0.00%
2	2.023005	1.90	95.00%	0.47%
4	1.148781	3.35	83.75%	5.08%
8	0.963380	3.99	49.87%	0.74%
16	0.712136	5.41	33.81%	16.09%

Q3: Distributed Dot Product Speedup



Analysis & Key Inferences

The distributed dot product demonstrates how to structure a program for massive datasets to achieve genuine Strong Scaling.

- **Inference 1: Data Locality Yields True Speedup.** Unlike DAXPY, this workload bypasses the $O(P)$ root memory bottleneck by generating vector chunks locally. Because the arithmetic phase is dense (3.85s baseline), the computation to communication ratio is high enough to yield a 5.41x speedup at 16 processes.
- **Inference 2: Amdahl's Law Dictates Efficiency Ceilings.** Despite the speedup, efficiency drops steadily from 95% down to 33.81%. This confirms Amdahl's Law: the maximum theoretical speedup is strictly bounded by the sequential fraction of the code. The initial broadcast and the final MPI_Reduce act as an unbreakable sequential ceiling, causing communication overhead to eventually climb back to 16.09% at $P=16$.
- **System Design Takeaway:** Maintaining high data locality (keeping data on the node that computes it) is the most effective strategy for large scale computing, though synchronization phases will eventually cap scalability.

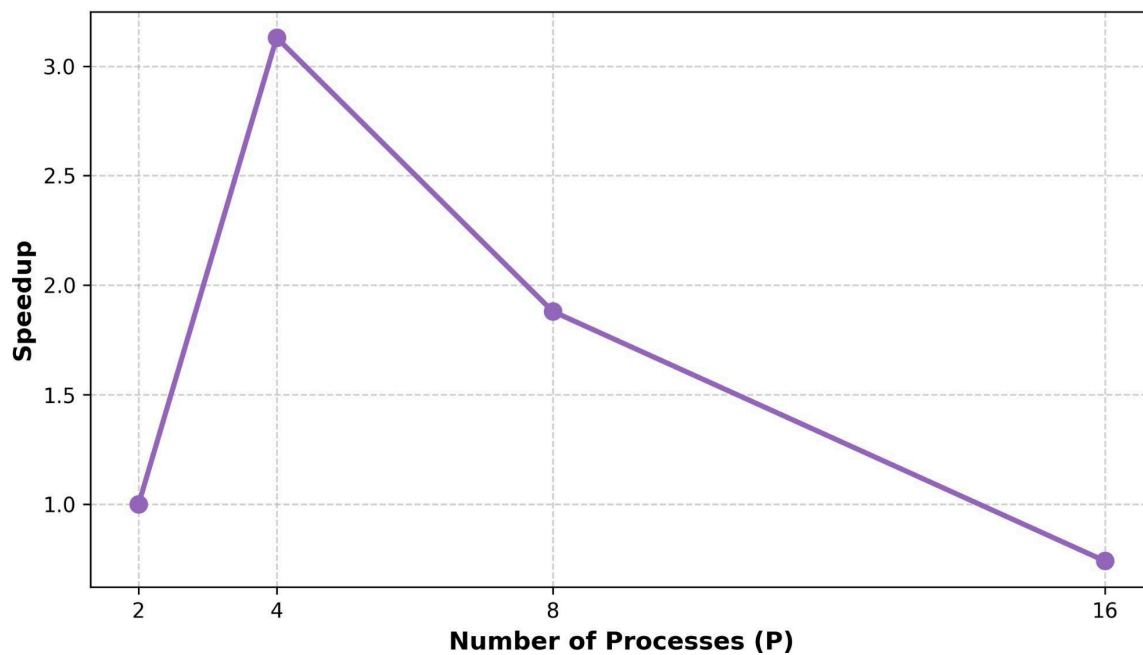
Q4: Master Slave Prime Search

Objective: Find all positive primes utilizing dynamic load balancing via MPI_ANY_SOURCE.

Performance Data :

Processes	Time (s)	Speedup	Efficiency
2 (1M+1S)	0.114727	1.00	100.00%
4 (1M+3S)	0.036631	3.13	78.25%
8 (1M+7S)	0.061030	1.88	23.50%
16 (1M+15S)	0.153971	0.74	4.62%

Q4: Master-Slave Prime Speedup



Analysis & Key Inferences

The Master Slave paradigm handles irregular workloads, but exposes the dangers of improper task granularity.

- **Inference 1: Dynamic Balancing Solves Computational Imbalance.** Primality testing is computationally uneven. The dynamic request system elegantly handles this up to 4 processes, achieving an excellent 3.13x speedup by ensuring no node sits idle.
- **Inference 2: Fine Grained Granularity Creates Message Storms.** The severe efficiency degradation at 8 processes, worsening at 16 processes (0.153s), exposes a critical architectural flaw. Sending a single integer per request creates an overwhelming storm of small network messages. We can infer that the Master node becomes completely CPU bound just processing MPI_Recv handlers, unable to feed the 15 starving slaves fast enough.
- **System Design Takeaway:** To optimize dynamic load balancing, developers must employ coarse grained parallelism. Sending blocks of 1,000 numbers per request would drastically lower the communication frequency and restore scalability.

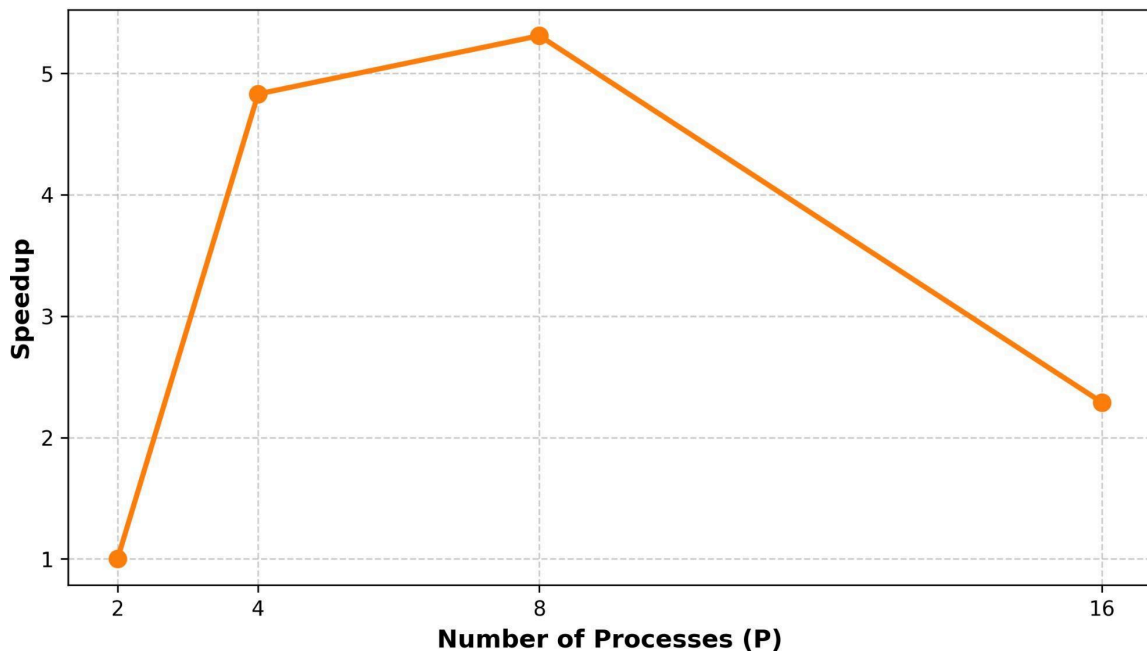
Q5: Master Slave Perfect Number Search

Objective: Find perfect numbers utilizing the Master Slave request pattern.

Performance Data :

Processes	Time (s)	Speedup	Efficiency
2 (1M+1S)	0.019043	1.00	100.00%
4 (1M+3S)	0.003941	4.83	120.75%
8 (1M+7S)	0.003585	5.31	66.37%
16 (1M+15S)	0.008289	2.29	14.31%

Q5: Perfect Number Speedup



Analysis & Key Inferences:

The perfect number search reinforces the limits of fine grained Master Slave architectures under heavier mathematical loads.

- **Inference 1: Higher Complexity Delays the Network Bottleneck.** Finding perfect numbers has an $O(\sqrt{N})$ complexity. Because the arithmetic weight per integer is heavier than prime searching, the slaves spend more time computing and less time communicating. Consequently, the architecture successfully scales up to 8 processes before stalling, unlike the prime search which stalled at 4.
- **Inference 2: Serialization Limits are Absolute.** Expanding to 16 processes triggers immediate negative scaling (0.0082 s). Regardless of the mathematical complexity, the Master node is ultimately forced to sequentially process individual network messages from 15 separate slaves. The time spent in context switching strictly overrides the time saved by distributed arithmetic.
- **System Design Takeaway:** Parallelization cannot overcome fundamental network serialization limits if the communication architecture requires $O(P)$ management from a single node. Batching tasks is mandatory for high core count clusters.